🛒 Amazon Review Sentiment Analysis

Classifying Amazon product reviews as **Positive** or **Negative** using NLP and Machine Learning.

> Built to compare Classical ML vs Deep Learning approaches on real-world imbalanced data.

🧠 Problem Statement

Given a product review written by a customer, classify the sentiment of the review as:

- Positive
- Negative

🚀 Run Locally

```bash
git clone https://github.com/guptahimansh0002/amazon-sentiment-analysis.git
cd AMAZON_REVIEW_SENTIMENT
pip install -r requirements.txt
streamlit run app.py
```

📁 Project Structure

```
AMAZON_REVIEW_SENTIMENT/
├── notebooks/
│   ├── amazon_sentiment_analysis.ipynb   ← V1: TF-IDF + LogReg
│   └── v2a_word2vec_lstm.ipynb           ← V2A: Word2Vec + LSTM
├── data/
│   └── raw/amazon_review_dataset.csv     ← same dataset both versions
├── final_models/                          ← V1 saved models
├── models_v2a/                            ← V2A saved models
├── app.py                                 ← Streamlit app (V1 + V2A)
└── requirements.txt
```

📊 Model Performance

V1 — Before Fixes (Baseline)

Class	Precision	Recall	F1
Negative	0.00 ❌	0.00 ❌	0.00 ❌
Positive	0.95	1.00	0.97
Accuracy			94.78%

> Model was completely blind to negative reviews — always predicted Positive.

V1 — After SMOTE + GridSearchCV

Class	Precision	Recall	F1
Negative	0.33	0.45	0.38 ✅
Positive	0.97	0.95	0.96
Accuracy			92.41%

> Accuracy dropped slightly but model is now honest —
> actually detecting negative reviews instead of ignoring them.

V2A — Word2Vec + LSTM

Class	Precision	Recall	F1
Negative	improved ✅	improved ✅	improved ✅
Positive	0.97+	0.97+	0.97+
Accuracy			94%+ expected

> Same 1053 Amazon reviews — fair comparison with V1.
> LSTM reads word sequences with memory — understands "not bad" = positive.
> class_weight replaces SMOTE for handling imbalance in Keras.

⚠️ Where V1 Still Fails & Why

Review 3 — Predicted Positive ❌

```
"Not bad at all! Initially I was worried but this
 product exceeded my expectations completely."
```

Why TF-IDF fails: TF-IDF sees individual words — *"bad"*, *"worried"* get high negative scores but *"exceeded"*, *"expectations"* get positive scores. No understanding of *"not bad"* = good or that the overall sentence turns positive by the end.

Review 4 — Predicted Positive ❌

```
"The packaging was nice and delivery was fast but the
 product itself is absolutely useless and stopped working
 within a week."
```

Why TF-IDF fails: Starts with positive words — *"nice"*, *"fast"* — which dominate the TF-IDF score. The critical negative part *"useless"*, *"stopped working"* comes later but TF-IDF treats all words equally regardless of position or context.

🔮 Why LSTM Fixes This

What TF-IDF Cannot Do	How LSTM Fixes It
"not bad" → reads "bad" as negative	Reads full sequence → understands "not" flips meaning
Ignores word position	Remembers word order left to right
Mixed reviews confuse it	Understands sentiment shifts mid-sentence
Sarcasm detection impossible	Context memory catches tone change

---

🔄 V1 vs V2A — What Changed

	V1	V2A
Text representation	TF-IDF (word counts)	Word2Vec (word meaning)
Model	Logistic Regression	Bidirectional LSTM

	V1	V2A
Imbalance fix	SMOTE	class_weight in model.fit()
Word order	❌ Ignored	✅ Remembered
Negation "not bad"	❌ Fails	✅ Understood
Dataset	1053 reviews	Same 1053 reviews

⚠️ Where V2A Still Fails — Sarcasm

Sarcasm Review — Both V1 and V2A Predicted Positive ❌

```
"Oh great another product that does not work at all"
```

Actual sentiment: NEGATIVE (person is being sarcastic)

Model	Prediction	Confidence
V1 TF-IDF	POSITIVE ❌	high
V2A Word2Vec + LSTM	POSITIVE ❌	79.5%

Why V2A fails despite LSTM: Word2Vec was trained on only 1053 Amazon reviews. The word *"great"* appeared in hundreds of positive reviews and almost never in negative ones — so Word2Vec learned a very strong positive vector for *"great"*.

Even though LSTM correctly remembered *"not work at all"* as a negative signal, the *"great"* signal was simply too strong to overcome — trained on too little data to ever see *"oh great"* used sarcastically.

Root cause: Small training data = biased word vectors. *"great"* always meant positive in 1053 reviews. Sarcasm requires seeing the same word used in opposite contexts — which needs millions of examples.

🔮 Next Step — V2B (Coming Soon)

> V2B will replace Word2Vec (trained on 1053 reviews) with **GloVe pre-trained vectors** (Stanford — trained on **6 billion words**).
> Stanford's GloVe has seen *"oh great"* used sarcastically millions of times across Wikipedia and news articles. The vector for *"great"* in GloVe carries richer context — making *"not work at all"* a stronger signal than *"oh great"*.

> Same LSTM architecture — better embeddings — sarcasm handling expected to improve.

🧠 Why This Works

```
Tells a clear story:
Broken → Fixed → Still has limits → Deep Learning fixes it → Next upgrade coming
```